Instituto Tecnológico de Buenos Aires

Trabajo Práctico Especial: Pre-entrega

Despliegue y Gestión del Clúster de Kubernetes



Integrantes:

- **Arancibia Carabajal, Nicolas (Leg. Nº 64481)**
- **Paletta, Lautaro (Leg. Nº 64499)**
- **Ronda, Agustín (Leg. Nº 64507)**

1. Problemática y contexto

The Store, el canal de venta online de nuestra empresa, se despliega con un script que crea un clúster de Kind (Kubernetes in Docker) en una sola máquina. Este entorno está pensado para desarrollar, pero no para operar. Cada operación sobre el clúster se hace a mano y depende de quien la ejecute, ya que los pasos no están escritos en ningún lado. Además, no se permite agregar ni quitar nodos de un clúster existente, así que hay que borrarlo y crearlo de nuevo, con la tienda fuera de servicio mientras tanto. Asimismo, si el tráfico crece, como en Black Friday o las ofertas de Navidad, no hay una forma rápida de sumar capacidad.

El problema, entonces, está en la plataforma. Lo que le falta a The Store es un clúster que cualquier integrante del equipo pueda crear, ampliar, reducir y reconstruir desde cero de forma automatizada y repetible.

2. Diseño de la solución

La plataforma tiene tres capas: los nodos, el clúster que corre sobre ellos y la aplicación que corre sobre el clúster. Ansible crea los nodos, instala el clúster y despliega la aplicación; desde ese momento Kubernetes mantiene la aplicación corriendo y reparte los pods entre los nodos disponibles.

- **Nodos.** Cada nodo es un contenedor Docker creado a partir de la imagen oficial de K3s. Los contenedores se conectan a una red de Docker propia, k3s-net, y cada uno recibe la IP fija que indica el inventario.
- **Clúster.** K3s, distribución liviana de Kubernetes. Un nodo corre como server, el control plane, y los demás como agents (workers).
- **Aplicación.** The Store, desplegada a partir de los manifiestos del repositorio, con las imágenes publicadas en un registry.
- **Automatización.** Un inventario de Ansible define qué nodos existen, con su nombre, rol e IP. Cada operación sobre el clúster es un playbook que lee ese inventario.

Hay playbooks para crear el clúster, desplegar The Store, agregar y quitar nodos, y destruir el entorno completo.

Decisiones técnicas

- **Ansible en lugar de un script:** Ansible compara el estado actual y modifica solo lo necesario, a diferencia de un script, que ejecuta siempre los mismos pasos. Por eso volver a correr un playbook sobre un clúster que ya está en el estado correcto no cambia nada, y agregar un nodo no toca a los que ya existen. Ansible trabaja sobre el daemon de Docker del host para crear y eliminar los nodos, y sobre la API de Kubernetes para desplegar la aplicación, eliminar nodos y verificar su estado.
- **Nodos como contenedores en lugar de máquinas virtuales:** Un contenedor arranca en segundos y consume una fracción de lo que consume una VM, así que podemos crear y eliminar nodos varias veces durante la demo en una sola notebook. Cada nodo corre en modo privilegiado, que K3s necesita para administrar la red y los recursos de los pods. La creación de los nodos está en un rol de Ansible separado de la instalación de K3s, así que se podría reemplazar por máquinas virtuales sin reescribir el resto de los playbooks.
- **Un registry en lugar de imágenes locales:** Actualmente, al desplegar el sistema, se cargan las imágenes directamente en el nodo único con kind load, algo que solo existe en Kind. En un clúster, cada nodo tiene que poder conseguirlas por su cuenta, por lo tanto las publicamos en GitHub Container Registry y cada uno las descarga cuando las necesita. Así, un nodo recién agregado no depende de que el playbook le copie nada.

- **Traefik como entrada a la tienda:** Usamos Traefik porque K3s lo instala e integra con ServiceLB, que publica el puerto 80 en cada nodo; ingress-nginx, que ya se estaba usando en The Store, exigiría desactivarlo, instalarlo y exponerlo aparte. Desde el host, la tienda se abre en el navegador con la IP de cualquier nodo de la red k3s-net.

3. Scope del POC y casos de uso

1 · Reproducibilidad: levantar el clúster desde cero.

- **Escenario:** Host con Docker, sin contenedores ni redes del proyecto. El inventario define un server y dos agents.
- **Acción:** Ejecutamos el playbook de creación del clúster y después lo ejecutamos una segunda vez.
- **Qué se observa:** Ansible crea la red y los tres contenedores, que arrancan K3s como server o como agent, y espera a que los tres nodos estén listos. En la segunda ejecución no encuentra nada que cambiar.
- **Criterio de éxito:** docker ps muestra los tres contenedores, kubectl get nodes muestra los tres nodos en estado Ready, y el resumen de Ansible muestra tareas con cambios en cada nodo en la primera ejecución y en la segunda ejecución no se ven cambios.

2 · Validación: desplegar The Store sobre el clúster.

- **Escenario:** Clúster del caso 1, sin la aplicación desplegada.
- **Acción:** Ejecutamos el playbook de despliegue de The Store.
- **Qué se observa:** Ansible aplica los manifiestos de The Store y espera a que todos los servicios estén disponibles. Cada nodo descarga del registry las imágenes de los pods que le tocan.
- **Criterio de éxito:** kubectl get pods -o wide muestra todos los pods en estado Running, repartidos entre los nodos, y la tienda abre en el navegador.

3 · Capacidad: sumar un nodo.

- **Escenario:** Clúster del caso 2, con la tienda funcionando.
- **Acción:** Agregamos un nuevo nodo worker al inventario y ejecutamos el playbook para agregar nodos.
- **Qué se observa:** Ansible crea solo el contenedor nuevo y lo une al clúster. Los nodos que ya existían figuran sin cambios.
- **Criterio de éxito:** docker ps muestra el contenedor nuevo y kubectl get nodes lista cuatro nodos en estado Ready.

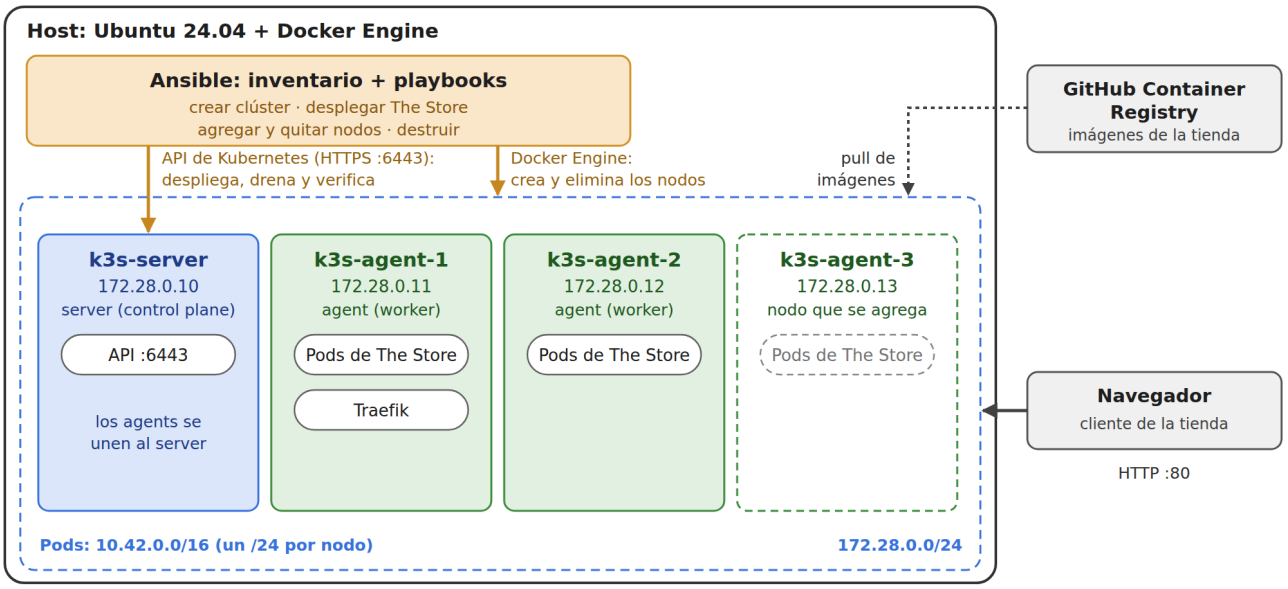
4 · Mantenimiento: retirar un nodo.

- **Escenario:** Clúster de cuatro nodos del caso 3.
- **Acción:** Ejecutamos el playbook para quitar nodos indicando uno de los workers y después lo sacamos del inventario.
- **Qué se observa:** Ansible marca el nodo para que no reciba pods nuevos y empieza a desalojar sus pods, y Kubernetes vuelve a crear esos pods en los demás nodos. Después Ansible borra el nodo del clúster y elimina su contenedor.
- **Criterio de éxito:** El nodo retirado ya no aparece en kubectl get nodes ni en docker ps, sus pods corren en otros nodos y la tienda sigue abriendo en el navegador.

5 · Reconstrucción: destruir el clúster y volver a crearlo.

- **Escenario:** Clúster funcionando, con la tienda desplegada.
- **Acción:** Ejecutamos el playbook de destrucción y después los de creación del clúster y despliegue de The Store.
- **Qué se observa:** El primer playbook elimina los contenedores, la red y los volúmenes del proyecto. Los otros dos vuelven a levantar el clúster y la tienda desde cero con el mismo inventario.
- **Criterio de éxito:** Después del destroy, docker ps, docker network ls y docker volume ls no muestran nada del proyecto. Después de la reconstrucción, el clúster y la tienda quedan como al final de los casos 1 y 2.

4. Diagrama de arquitectura



5. Alternativas consideradas

- **kops en AWS:** Crea máquinas, red y clúster en AWS y trae escalado, rolling update y backup a S3 como comandos propios. Se descartó no solo por el costo, sino también porque dependeríamos de un único proveedor. Además, kops automatiza la red y las máquinas y disminuye el rol de Ansible en la solución.
- **Minikube:** Con un solo comando crea las máquinas virtuales e instala Kubernetes, pero lo resuelve todo por su cuenta, usa su propio sistema operativo y arma su propia red. Eso nos impediría diseñar la red y elegir el sistema operativo, y dejaría a Ansible sin nada que automatizar.
- **Kind y k3d:** Kind es lo que usa hoy local.sh y no permite agregar ni quitar nodos de un clúster existente. k3d corre K3s en contenedores Docker, igual que nuestra solución, pero crea y une los nodos con sus propios comandos, y Ansible quedaría reducido a llamar a k3d.
- **Máquinas virtuales con Multipass o en la nube:** Fue nuestra primera opción, con los mismos playbooks y Ansible conectándose a cada VM por SSH. La descartamos porque crear una VM tarda minutos y cada nodo necesita al menos 2 GB de RAM, lo que vuelve lentas las operaciones de agregar y quitar nodos en una notebook. En la nube, además, se suma el costo de tener al menos tres VMs funcionando.